

FUNCDB – MINI SISTEMA GERENCIADOR DE BANCO DE DADOS EM HASKELL

Universidade Federal de Roraima (UFRR)
Departamento de Ciência da Computação (DCC)
Disciplina: COMP914 – Programação Funcional
Professor: Jean Bertrand

1. Introdução

A Programação Funcional é um paradigma de desenvolvimento de software baseado no uso de funções matemáticas, imutabilidade de dados e avaliação declarativa. Diferentemente da programação imperativa, onde o foco está na alteração de estados e execução de comandos sequenciais, a programação funcional busca descrever o que deve ser feito, e não necessariamente como deve ser feito.

A linguagem Haskell é uma das principais representantes desse paradigma, oferecendo tipagem estática forte, avaliação preguiçosa (lazy evaluation), funções de alta ordem e mecanismos robustos para tratamento seguro de erros.

Neste contexto, foi desenvolvido o projeto **FuncDB**, um Mini Sistema Gerenciador de Banco de Dados (Mini-SGBD) capaz de realizar operações básicas inspiradas na linguagem SQL, como inserção, consulta, atualização, remoção e ordenação de registros.

O objetivo principal do projeto é demonstrar a aplicação prática dos conceitos estudados ao longo da disciplina, atendendo aos requisitos técnicos estabelecidos para o projeto final.

2. Objetivos

2.1 Objetivo Geral

Desenvolver um Mini Sistema Gerenciador de Banco de Dados utilizando a linguagem Haskell, aplicando os principais conceitos da Programação Funcional.

2.2 Objetivos Específicos

- Implementar tipos customizados utilizando sintaxe de registros.
 - Utilizar persistência de dados em disco.
 - Aplicar tratamento seguro de erros com Maybe e Either.
 - Empregar funções de alta ordem como map, filter e fold'.
 - Implementar algoritmos de Divisão e Conquista.
 - Simular operações de um banco de dados relacional.
-

3. Fundamentação Teórica

3.1 Programação Funcional

A Programação Funcional é baseada no conceito de funções puras, ou seja, funções que produzem sempre o mesmo resultado para uma mesma entrada e não geram efeitos colaterais.

Esse paradigma favorece:

- Modularidade;
- Reutilização de código;
- Facilidade de manutenção;
- Redução de erros.

A linguagem Haskell foi projetada especificamente para explorar esses conceitos.

3.2 Sistemas Gerenciadores de Banco de Dados

Um Sistema Gerenciador de Banco de Dados (SGBD) é um software responsável por armazenar, organizar e recuperar informações estruturadas.

Entre as funcionalidades mais comuns de um SGBD estão:

- Criação de tabelas;
- Inserção de registros;
- Consultas;
- Atualizações;
- Exclusões;
- Ordenação de dados.

O FuncDB implementa uma versão simplificada dessas operações.

4. Estruturas de Dados Utilizadas

O sistema foi modelado utilizando tipos algébricos da linguagem Haskell.

Tipo

Representa os tipos possíveis de uma coluna.

```
data Tipo
  = TTexto
  | TInteiro
  | TBooleano
```

Célula

Representa os valores armazenados em uma tabela.

```
data Celula
  = Texto String
  | Inteiro Int
  | Booleano Bool
```

Tabela

Representa uma tabela do banco de dados.

```
data Tabela = Tabela
  { nomeTabela :: String
  , esquema :: [(String, Tipo)]
  , linhas :: [Linha]
  }
```

Essa implementação atende ao requisito de utilização de tipos customizados avançados com registros nomeados.

5. Persistência de Dados

O projeto utiliza a mônada IO para realizar operações de entrada e saída.

Os dados são armazenados no arquivo:

banco.txt

Ao iniciar o programa, o sistema tenta carregar automaticamente os dados previamente armazenados.

Ao encerrar a execução, o estado atual do banco é salvo novamente no arquivo.

Essa funcionalidade garante persistência entre diferentes execuções do programa.

6. Tratamento Seguro de Erros

Uma das exigências do projeto era evitar funções parciais que pudessem causar falhas inesperadas.

Para isso foram utilizados os tipos Maybe e Either.

Maybe

Utilizado em operações que podem ou não encontrar um resultado.

Exemplo:

```
buscarTabela :: String -> Banco -> Maybe Tabela
```

Caso a tabela exista, retorna:

Just tabela

Caso contrário:

Nothing

Either

Utilizado para retornar sucesso ou erro explicitamente.

Exemplo:

```
inserirLinha :: Linha -> Tabela -> Either String Tabela
```

Se a operação for válida:

Right tabela

Caso contrário:

Left "Mensagem de erro"

Esse mecanismo evita encerramentos inesperados do sistema.

7. Funções de Alta Ordem

O projeto utiliza explicitamente as funções exigidas pelo enunciado.

Map

Utilizada para transformar coleções de dados.

Exemplo:

listarColunaTexto

O map percorre todas as linhas de uma tabela e extrai apenas os valores desejados.

Filter

Utilizada para consultas.

Exemplo:

selectWhereInteiroMaior

O filter mantém apenas os registros que satisfazem determinada condição.

Foldl'

Utilizada para operações de agregação.

Exemplo:

somarColuna

Essa função percorre toda a tabela acumulando valores para produzir um resultado final.

8. Algoritmo de Divisão e Conquista

Para atender ao requisito de utilização de algoritmos de Divisão e Conquista foi implementado o algoritmo Merge Sort.

O Merge Sort divide recursivamente uma lista em duas metades até que cada sublista contenha apenas um elemento.

Posteriormente as sublistas são combinadas de forma ordenada.

Etapas:

1. Divisão da lista.
2. Ordenação recursiva.
3. Combinação das listas ordenadas.

Complexidade:

- Melhor caso: $O(n \log n)$
- Caso médio: $O(n \log n)$
- Pior caso: $O(n \log n)$

Essa implementação é utilizada no comando ORDER.

9. Funcionalidades Implementadas

O sistema disponibiliza as seguintes operações:

Gerenciamento de Tabelas

- CREATE TABLE
- DROP TABLE
- SHOW TABLES

Manipulação de Dados

- INSERT
- UPDATE
- DELETE

Consultas

- SELECT
- WHERE
- ORDER BY

Agregações

- SUM
- AVG
- COUNT

Persistência

- SAVE
 - LOAD
-

10. Casos de Teste

Durante o desenvolvimento foram realizados diversos testes para validar o comportamento do sistema.

Inserção

```
INSERT Usuario Pedro 22 True
```

Resultado:

Registro inserido com sucesso.

Consulta

```
SELECT * FROM Usuarios WHERE idade > 18
```

Resultado:

Retorno apenas dos usuários maiores de idade.

Ordenação

ORDER Usuarios BY idade

Resultado:

Tabela ordenada corretamente utilizando Merge Sort.

Persistência

SAVE

EXIT

Após reiniciar o sistema:

LOAD

Resultado:

Os dados permanecem armazenados.

11. Conclusão

O projeto FuncDB demonstrou a aplicação prática dos principais conceitos da Programação Funcional através da implementação de um Mini Sistema Gerenciador de Banco de Dados.

Foram utilizados tipos customizados, persistência em disco, tratamento seguro de erros, funções de alta ordem e algoritmos de Divisão e Conquista, atendendo integralmente aos requisitos estabelecidos para o projeto final.

Além disso, o sistema apresenta funcionalidades inspiradas em bancos de dados reais, proporcionando uma experiência prática de desenvolvimento utilizando Haskell.

12. Referências

HUTTON, Graham. Programming in Haskell. Cambridge University Press, 2016.

LIPOVAČA, Miran. Learn You a Haskell for Great Good! No Starch Press, 2011.

THOMPSON, Simon. Haskell: The Craft of Functional Programming. Addison-Wesley, 2011.

BIRD, Richard. Introduction to Functional Programming using Haskell. Prentice Hall, 1998.